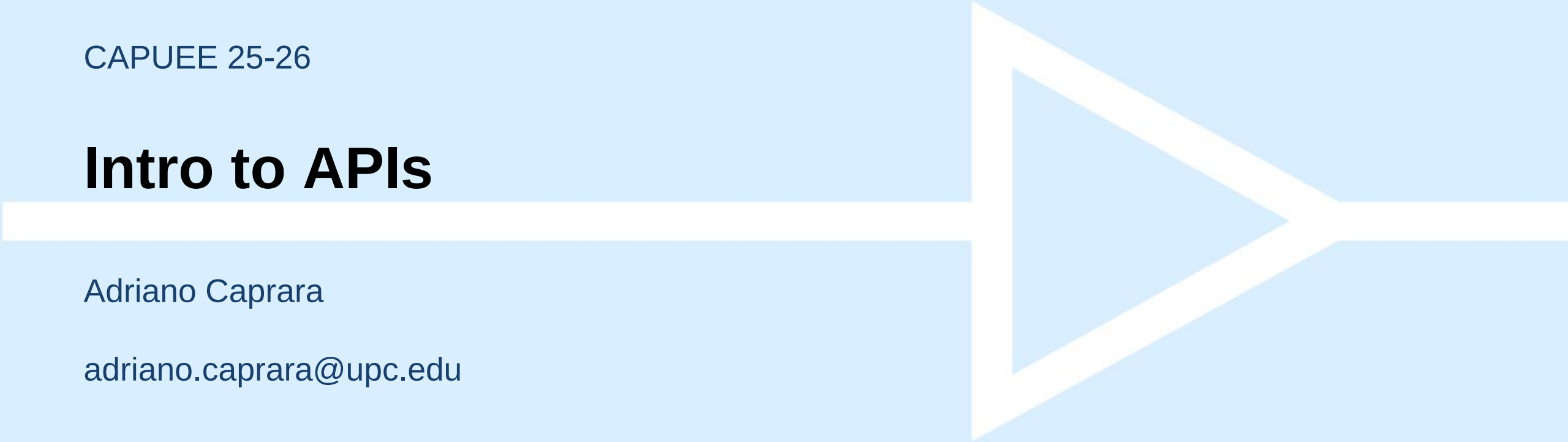


CAPUEE 25-26

Intro to APIs



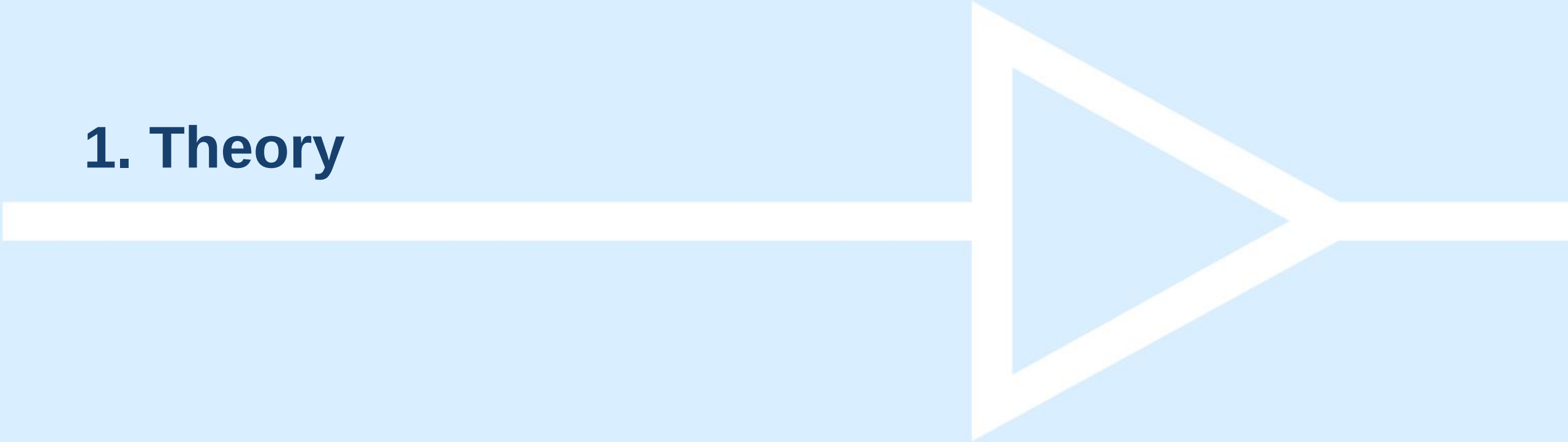
Adriano Caprara

adriano.caprara@upc.edu

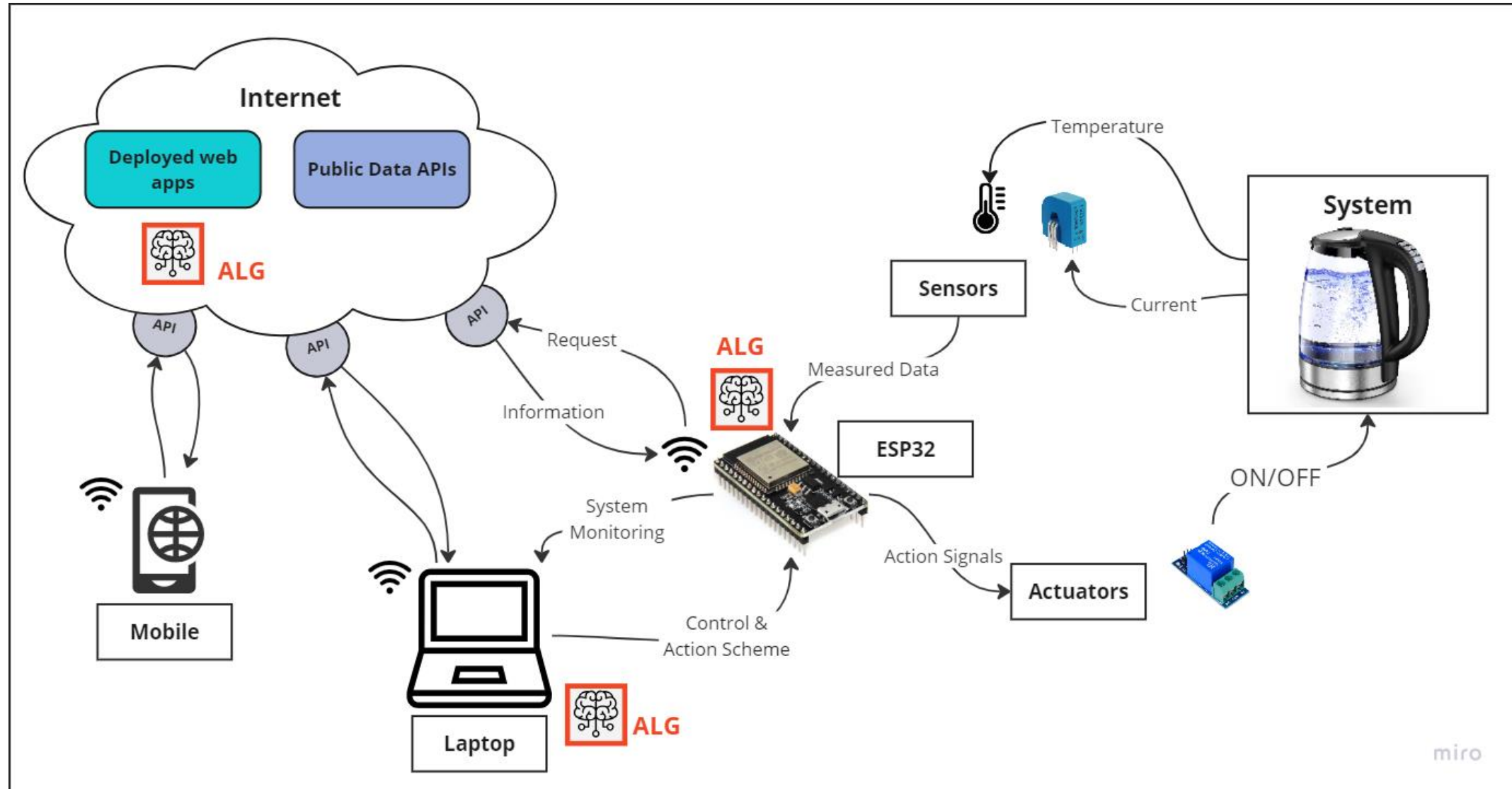
Agenda

1. APIs introduction
2. API examples
3. Market prices from REE API
4. REE API exercise

1. Theory

A large, stylized graphic on the right side of the slide. It consists of a thick, light blue horizontal line that enters a large, white-outlined triangle from the left. The triangle is filled with a light blue color, matching the background. The line exits the triangle on the right side, continuing as a horizontal line.

Communications & data flows



What is an API?

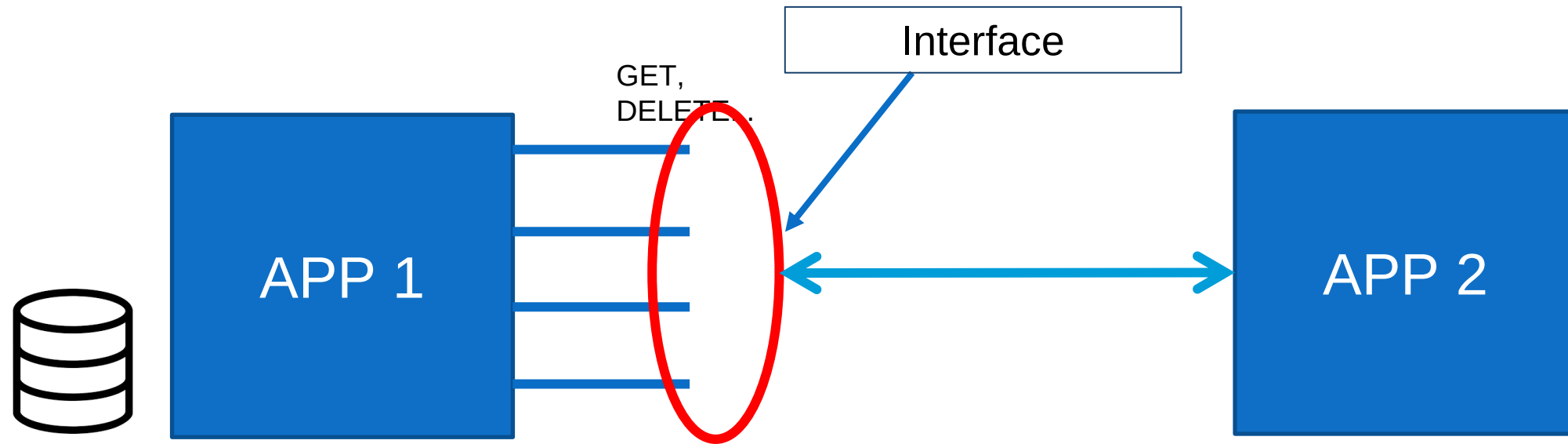
API: Application Programming Interface

It is a formalized contract. It is a **set of rules** and tools that allows different software programs to communicate with each other.

It defines the:

- **What** you can **request** from a system
- **How** to request it
- **What** you will **receive** back from the request
- **If** you have the authorization to get this data (i.e. users, access tokens...)

What is an API?



APIs hide internal complexity: Implementation details (algorithms, data storage, hardware control) and resources management are not seen by the user

Restaurant analogy:

- the kitchen is the APP I want to access and its data
- the menu is the of rules and resources that I, the customer want to Access
- the API is the **waiter**, that only brings me the ítems on the menu (and ofc if we speak the same language)

Restaurants vs APIs (advanced)

RESTAURANT

Kitchen

Menu

Waiter

Customer

Order

Dish served

Restaurant hours

Reservation

API

Application/server

API endpoints (and documentation)

API interface

Client

API request (i.e. GET)

API response (i.e. data retrieved)

Rate limits (i.e. Max #calls/day)

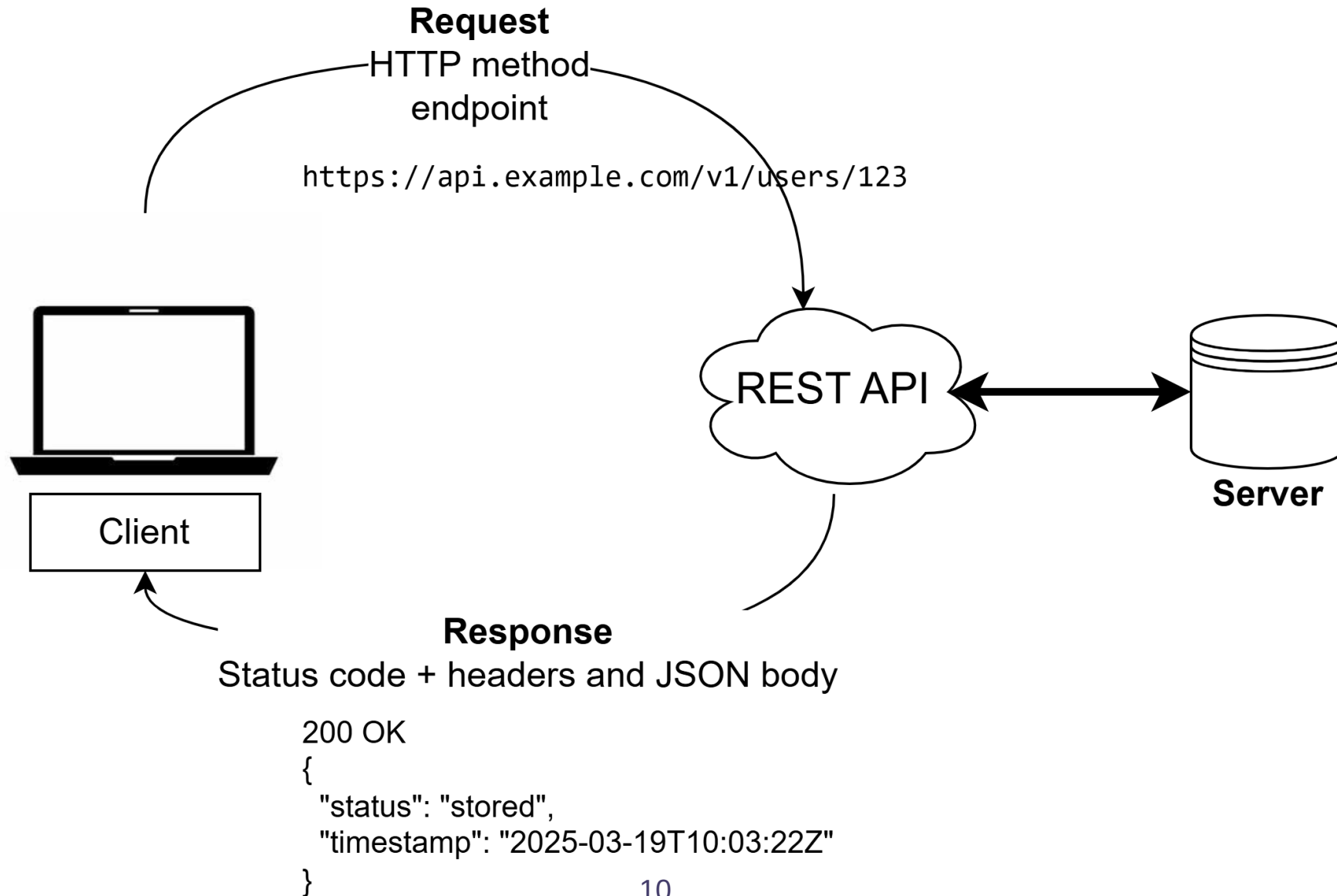
Credentials & authentication

REST APIs

A Web-Based API Using Standard Web Rules (HTTP protocol)

- REST = REpresentational State Transfer
- It is a **stateless** client-server communication, meaning no client information is stored between get requests and each **request** is **separate** and **unconnected**
- APIs with REST architecture are fast and lightweight to be created, with great scalability, perfect for IoT and mobile app development. That's why their success and large adoption
- Uses the HTTP protocol (the same that powers our web)
- Responses are generally formatted as either JSON or XML to transmit the data.
The structure is normally detailed and accessible in the API documentation

API communication scheme: request and response



1) Methods

- GET: retrieve something
- POST: create something
- PUT: replace (i.e. Edit a comment)
- DELETE: delete something (i.e. User wants to delete his/her information)

2) Endpoints (i.e. The address of the operation)



3) Headers (metadata about the request)

- Authentication (i.e. Access token)
- Format (typically json, but also xml and other formats possible)

4) Body (optional)

Additional info to be sent to the server for some types of requests (i.e. POST)

Response

1) Status code (mandatory) -> <https://http.cat/>

- 2xx (success)
- 4xx (Client error)
- 5xx (Server error)

2) Headers (metadata about the response)

3) Body (usually JSON if we request data)

Goodbye JSON?

```
[  
  { "id": 1, "name": "Alice", "role": "admin" },  
  { "id": 2, "name": "Bob", "role": "user" }  
]
```

```
users[2]{id,name,role}:  
  1,Alice,admin  
  2,Bob,user
```

TOON (Toke-Oriented Object Notation) needs less characters. This means fewer tokens for the LLMs and increased computational efficiency just by improving the format

APIs: the bigger picture

- Modern APIs adhere to **standards** (typically HTTP and REST), that are developer-friendly, easily accessible and understood broadly
- They are treated **more like products than code**. They are designed for consumption for specific audiences (e.g., mobile developers), they are documented, and they are versioned in a way that users can have certain expectations of its maintenance and lifecycle
- Because they are much more standardized, they have a much stronger discipline for **security** and governance, as well as monitored and **managed for performance and scale**
- As any other piece of productized software, the modern API has its **own software development lifecycle** (SDLC) of designing, testing, building, managing, and versioning. Also, modern APIs are well documented for consumption and versioning

- You want to **protect your APIs** from overuse and abuse, so you use an **authentication service** and rate limiting.
- You want to understand how people use your APIs, so you've added **analytics** and **monitoring** tools.
- If you have **monetized APIs**, you'll want to connect to a **billing system**.

Things can get even more complex...

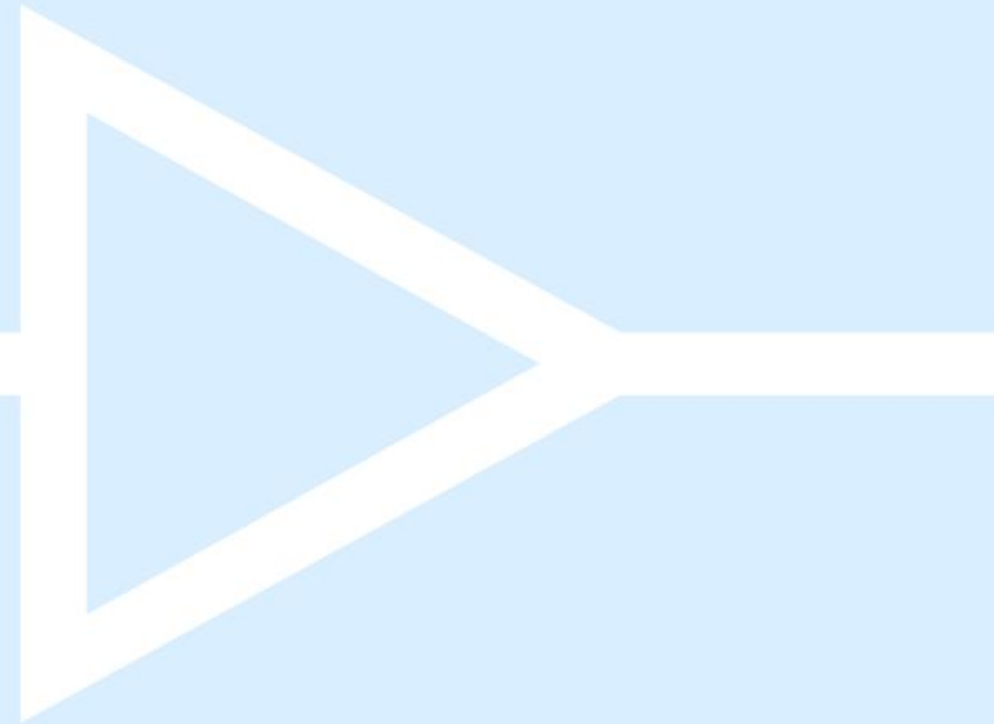
- You may have adopted a **microservices architecture** for your application, in which case a single request could require calls to dozens of distinct applications.

The API Gateway

An API gateway is an **API management tool** that sits between a client and a collection of backend services.



2. Practice



“I’m not afraid of artificial intelligence. I’m afraid of human stupidity which is natural.”

- Cosimo Fini

<https://pokeapi.co/>

<https://portal.electricitymaps.com/dashboard>

<https://www.ree.es/es/datos/apidatos>

THANK YOU FOR YOUR ATTENTION



TECHNOLOGY CENTER

UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

